

Lab05 pipelined CPU - Basic implimentation

2024084648 이종민

1. Impliment

(1) pipeline

CTRL.v에서 명령어를 확인해서 EX, MEM, WB단계에 필요한 신호를 나눠준다.

한 사이클에 모든 신호를 결정해야 하므로 single cycle CPU에서의 CTRL.v를 이용했다.

```
66 // I-Type
67 `OP_ADDIU: begin ALUSrc = 1; SignExtend = 1; RegWrite = 1; ALUOp = `ALU_ADDU; end
68 `OP_ANDI: begin ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_AND; end
69 `OP_ORI: begin ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_OR; end
70 `OP_XORI: begin ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_XOR; end
71 `OP_LUI: begin ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_LUI; end
72 `OP_SLTI: begin ALUSrc = 1; SignExtend = 1; RegWrite = 1; ALUOp = `ALU_SLT; end
73 `OP_SLTIU: begin ALUSrc = 1; SignExtend = 1; RegWrite = 1; ALUOp = `ALU_SLTU; end
74 `OP_LW: begin ALUSrc = 1; SignExtend = 1; RegWrite = 1; MemRead = 1; MemtoReg = 0; ALUOp = `ALU_ADDU; end
75 `OP_SW: begin ALUSrc = 1; SignExtend = 1; MemWrite = 1; ALUOp = `ALU_ADDU; end
76 `OP_BEQ: begin SignExtend = 1; Branch = 1; ALUOp = `ALU_EQ; end
77 `OP_BNE: begin SignExtend = 1; Branch = 1; ALUOp = `ALU_NEQ; end
78
79 // J-Type
80 `OP_J: begin Jump = 1; end
81 `OP_JAL: begin Jump = 1; RegWrite = 1; SavePC = 1; end
82
83 default: begin end
84 endcase
85
86 if (PCSrc) begin
87   RegWrite = 0;
88   MemWrite = 0;
89 end
90
91 EX = {ALUOp, ALUSrc, RegDst};
92 MEM = {Branch, MemRead, MemWrite};
93 WB = {SavePC, RegWrite, MemtoReg};
94 end
```

CPU.v에서 ID/EX, EX/MEM, MEM/WB단계 사이에서의 래치와 와이어를 선언한다. Positive edge clock마다 래치 값을 업데이트해주었다.

그리고 stage에 맞는 control signal을 연결했다.

```
PC <= PC_plus4;
IR <= inst;
IFtoID_nextPC <= PC_plus4;

IDtoEX_WBreg <= WB;
IDtoEX_MEMreg <= MEM;
IDtoEX_EXreg <= EX;
IDtoEX_ext <= ext_imm;
IDtoEX_nextPC <= IFtoID_nextPCwire;
IDtoEX_rd_data1 <= rd_data1;
IDtoEX_rd_data2 <= rd_data2;
IDtoEX_rt <= rt;
IDtoEX_rd <= rd;
IDtoEX_shamt <= shamt;
IDtoEX_halt <= (IRwire == 0);

ExtoMEM_WBwire <= IDtoEX_WBwire;
ExtoMEM_MEMreg <= IDtoEX_MEMwire;
ExtoMEM_ADDresult <= addResultWire;
ExtoMEM_zero <= zero;
ExtoMEM_ALUresult <= alu_result;
ExtoMEM_wrd_data <= IDtoEX_rd_data2Wire;
ExtoMEM_PCplus4 <= IDtoEX_nextPCwire;
ExtoMEM_destination <= IDtoEX_destinationWire;
ExtoMEM_halt <= IDtoEX_haltWire;

MEMtoWB_WBreg <= ExtoMEM_WBwire;
MEMtoWB_memorydata <= mem_read_data;
MEMtoWB_ALUresult <= ExtoMEM_ALUresultWire;
MEMtoWB_destination <= ExtoMEM_destinationWire;
MEMtoWB_PCplus4 <= ExtoMEM_PCplus4Wire;
MEMtoWB_halt <= ExtoMEM_haltWire;

// Control Signal Demux from Pipeline Wires
assign PCSrc = Branch & ExtoMEM_zeroWire;
assign {ALUOp, ALUSrc, RegDst} = IDtoEX_EXwire;
assign {Branch, MemRead, MemWrite} = ExtoMEM_MEMwire;
assign {SavePC, RegWrite, MemtoReg} = MEMtoWB_WBwire;
```

(2) data hazard (stall)

HAZARD.v에서 older instruction 중에 만약 레지스터 목적지가 rd_addr1이나 rd_addr2과 같으면서 regwrite = 1인 경우 stall condition을 1로 하고 나머지 경우는 0으로 하였고 이 stall 신호를 output reg로 cpu에 전달하였다.

CPU에서 stall이 상태에서 IF, ID stage 래치를 유지하고 이후 stage는 계속 진행시켜준다.

```
// ----- Case 2: Hazard Detected (Stall) -----
else if (stall) begin
    // Freeze IF stage
    PC <= PC;

    // Freeze IF/ID pipeline reg
    IR <= IR;
    IFtoID_nextPC <= IFtoID_nextPC;

    // Insert bubble into ID/EX
    IDtoEX_WBreg <= 3'b00;
    IDtoEX_MEMreg <= 3'b000;
    IDtoEX_EXreg <= 6'b000000;
    IDtoEX_ext <= 32'b0;
    IDtoEX_nextPC <= 32'b0;
    IDtoEX_rd_data1 <= 32'b0;
    IDtoEX_rd_data2 <= 32'b0;
    IDtoEX_rt <= 5'b0;
    IDtoEX_rd <= 5'b0;
    IDtoEX_shamt <= 0;

    // EX stage continues
    EXtoMEM_WBreg <= IDtoEX_WBwire;
    EXtoMEM_MEMreg <= IDtoEX_MEMwire;
    EXtoMEM_ADDresult <= addResultWire;
    EXtoMEM_zero <= zero;
    EXtoMEM_ALUresult <= alu_result;
    EXtoMEM_wrddata <= IDtoEX_rd_data2Wire;
    EXtoMEM_destination <= IDtoEX_destinationWire;
    EXtoMEM_PCplus4 <= IDtoEX_nextPCwire;
    EXtoMEM_halt <= IDtoEX_haltWire;

    // MEM stage continues
    MEMtoWB_WBreg <= EXtoMEM_WBwire;
    MEMtoWB_memorydata <= mem_read_data;
    MEMtoWB_ALUresult <= EXtoMEM_ALUresultWire;
    MEMtoWB_destination <= EXtoMEM_destinationWire;
    MEMtoWB_PCplus4 <= EXtoMEM_PCplus4wire;
    MEMtoWB_halt <= EXtoMEM_haltWire;
end
```

(3) branch predict not taken (flush)

```
// ----- Case 5: Normal Execution -----
else begin
    PC <= PC_plus4;
    IR <= inst;
    IFtoID_nextPC <= PC_plus4;
end
```

CPU에서 branch 여부에 관계없이 일단 PC plus 4로 진행시켜준다.

```
// Control Signal Demux from Pipeline Wires
assign PCSrc = Branch & EXtoMEM_zeroWire;
```

만약 MEM stage에서 branch가 taken인게 판단나면

younger instruction을 flush해줘야 한다.

따라서 해당 clock에 PC를 alu add result로 바꿔주고

해당하는 래치들을 0으로 해주고

IR은 무효한 값으로 바꿔준다. (0은 halt가 1이 되므로 다른 수를 사용했다)

```
// ----- Case 1: Flush Younger Instructions (Branch Taken) -----
else if (PCSrc) begin
    PC          <= EXtoMEM_ADDresultWire;
    IR          <= 32'hFC00_0000;

    IDtoEX_EXreg <= 6'b000000;
    IDtoEX_MEMreg <= 3'b000;
    IDtoEX_WBreg <= 3'b000;
    IDtoEX_halt  <= 0;

    EXtoMEM_WBreg <= 3'b000;
    EXtoMEM_MEMreg <= 3'b000;
    EXtoMEM_ADDresult <= 0;
    EXtoMEM_zero <= 0;
    EXtoMEM_ALUresult <= 0;
    EXtoMEM_wrddata <= 0;
    EXtoMEM_destination <= 0;
    EXtoMEM_PCplus4 <= 0;
    EXtoMEM_halt <= 0;

    MEMtoWB_WBreg <= EXtoMEM_WBwire;
    MEMtoWB_memorydata <= mem_read_data;
    MEMtoWB_ALUresult <= EXtoMEM_ALUresultWire;
    MEMtoWB_destination <= EXtoMEM_destinationWire;
    MEMtoWB_PCplus4 <= EXtoMEM_PCplus4wire;
    MEMtoWB_halt <= 0;
end
```

(4) J, Jal, Jr

Jump인지 판단은 ID stage에서 CTRL로부터 받을 수 있다. 따라서 IF stage에 있는 instruction 대신 jump할 곳의 주소를 PC에 저장해야하고 IR 래치를 초기화해준다.

이후 래치들은 평소와 같이 작동해준다.

```
// ---- Case 3: Jump Instruction ----
else if (Jump) begin
    PC          <= (IFtoID_nextPCwire[31:28], immj, 2'b00);
    IR          <= 32'hFC00_0000;
    IFtoID_nextPC <= PC_plus4;

    IDtoEX_WBreg <= WB;
    IDtoEX_MEMreg <= MEM;
    IDtoEX_EXreg <= EX;
    IDtoEX_ext   <= ext_imm;
    IDtoEX_nextPC <= IFtoID_nextPCwire;
    IDtoEX_rd_data1 <= rd_data1;
    IDtoEX_rd_data2 <= rd_data2;
    IDtoEX_rt    <= rt;
    IDtoEX_rd    <= rd;
    IDtoEX_shamt <= shamt;
    IDtoEX_halt   <= (IRwire == 0);

    EXtoMEM_WBreg <= IDtoEX_WBwire;
    EXtoMEM_MEMreg <= IDtoEX_MEMwire;
    EXtoMEM_ADDresult <= addResultWire;
    EXtoMEM_zero     <= zero;
    EXtoMEM_ALUresult <= alu_result;
    EXtoMEM_wrdata   <= IDtoEX_rd_data2Wire;
    EXtoMEM_Pcplus4  <= IDtoEX_nextPCwire;
    EXtoMEM_destination <= IDtoEX_destinationWire;
    EXtoMEM_halt     <= IDtoEX_haltWire;

    MEMtoWB_WBreg <= EXtoMEM_WBwire;
    MEMtoWB_memorydata <= mem_read_data;
    MEMtoWB_ALUresult <= EXtoMEM_ALUresultWire;
    MEMtoWB_destination <= EXtoMEM_destinationWire;
    MEMtoWB_Pcplus4 <= EXtoMEM_Pcplus4Wire;
    MEMtoWB_halt     <= EXtoMEM_haltWire;
end
```

Jal인 경우 CTRL에서 SavePC 신호를 WB stage 신호로 전달해준다.

SavePC == 1인 경우 31번째 레지스터에 PC+4를 저장해준다.

이때 PC + 4값을 WB stage에서 받기 위한 래치를 파이프라인에 추가했다.

Jr은 JUMP와 동일한 방식으로 하되 PC값만 rd_data1로 대입하도록 했다.

```
// ---- Case 4: Jump Register (JR) ----
else if (JR) begin
    PC          <= rd_data1;
    IR          <= 32'hFC00_0000;
    IFtoID_nextPC <= PC_plus4;
end
```

2. Trouble shooting

(1) Halt

테스트 벤치에서 halt가 1이 된 clock을 기준으로 프로그램을 끝내는데,

Halt인지 여부는 파이프라인의 IF stage에서 정해지므로 이후 stage들이 마무리 되지 않은 채 프로그램이 종료되는 문제가 있었다.

따라서 halt 값도 파이프라인에 포함시켜 MEM/WB 래치로부터 halt를 가져오도록 했다.

```
assign halt = MEMtoWB_halt;
```